

Unit Tests für PySpark (pytest, assertDataFrameEqual)

Section 5 · Implementing CI-CD · Quelle: Kurs 4, M02 Kapitel 9–10 + tests/unit_tests/

Unit-Tests prüfen einzelne Funktionen isoliert – ohne einen kompletten Pipeline-Lauf und ohne echte Produktionsdaten. Für PySpark-Code stellt Apache Spark seit Version 3.5 mit dem Modul `pyspark.testing.utils` zwei speziell auf DataFrames zugeschnittene Vergleichsfunktionen bereit: `assertDataFrameEqual` und `assertSchemaEqual`. In Kombination mit dem Python-Testframework **pytest** lassen sich damit die im vorigen Kapitel modularisierten Funktionen systematisch absichern.

1. Testaufbau mit einer pytest-Fixture

Da PySpark-Funktionen eine aktive `SparkSession` benötigen, wird diese üblicherweise einmal pro Testlauf über eine pytest-**Fixture** bereitgestellt und an alle Testfunktionen weitergereicht:

```
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType, DoubleType, LongType
from pyspark.sql.functions import when, col
from pyspark.testing.utils import assertDataFrameEqual, assertSchemaEqual

from src.helpers import project_functions

@pytest.fixture(scope="session")
def spark():
    spark = SparkSession.builder.getOrCreate()
    yield spark
```

2. Schema-Test mit assertSchemaEqual

Die Funktion `assertSchemaEqual` vergleicht ausschließlich das Schema zweier DataFrames (Spaltennamen, Datentypen, Nullable-Flag) – ideal, um sicherzustellen, dass eine Schema-

Definitionsfunktion wie `get_health_csv_schema()` nach künftigen Änderungen weiterhin das erwartete Format liefert:

```
def test_get_health_csv_schema_match():
    expected_schema = StructType([
        StructField("ID", IntegerType(), True),
        StructField("PII", StringType(), True),
        StructField("date", DateType(), True),
        StructField("HighCholest", IntegerType(), True),
        StructField("HighBP", DoubleType(), True),
        StructField("BMI", DoubleType(), True),
        StructField("Age", DoubleType(), True),
        StructField("Education", DoubleType(), True),
        StructField("income", IntegerType(), True)
    ])

    actual_schema = project_functions.get_health_csv_schema()
    assertSchemaEqual(actual_schema, expected_schema)
```

3. Datenvergleich mit `assertDataFrameEqual`

Um eine Mapping-Funktion wie `high_cholest_map` zu testen, erzeugt man einen kleinen, handgeschriebenen Beispiel-DataFrame mit bekannten Eingaben, wendet die zu testende Funktion darauf an, und vergleicht das Ergebnis mit einem ebenfalls handgeschriebenen erwarteten DataFrame:

```
def test_high_cholest_column_valid_map(spark):
    data = [(0,), (1,), (2,), (3,), (4,), (None,)]
    sample_df = spark.createDataFrame(data, ["value"])

    actual_df = sample_df.withColumn("actual",
    project_functions.high_cholest_map("value"))

    expected_df = spark.createDataFrame(
        [(0, "Normal"), (1, "Above Average"), (2, "High"),
        (3, "Unknown"), (4, "Unknown"), (None, "Unknown")],
        schema=StructType([
            StructField("value", LongType(), True),
            StructField("actual", StringType(), True)
        ])
    )

    assertDataFrameEqual(actual_df.select(col('actual')),
    expected_df.select(col('actual')))
```

Wichtig: Werden diese Tests über die Kommandozeile mit `pytest` ausgeführt (nicht als Notebook-Zelle), muss der Pfad zum `src`-Modul im `PYTHONPATH` verfügbar sein, damit `from src.helpers import project_functions` funktioniert – genau dieser Aufruf ist es auch, der später in einer CI-Pipeline automatisiert ausgeführt wird (siehe Kapitel 4).

Ergänzt aus der offiziellen Databricks-Dokumentation/Apache Spark: `assertDataFrameEqual`

unterstützt zahlreiche optionale Parameter, um den Vergleich flexibel zu steuern – u. a.

`checkRowOrder` (Zeilenreihenfolge ignorieren), `rtol` / `atol` (Toleranzen für Fließkommazahlen), `ignoreColumnOrder` und `ignoreColumnType`. Bei sehr großen DataFrames kann die Funktion laut Databricks-Knowledgebase zu `OutOfMemoryError` führen, da für den Vergleich Daten auf den Treiber gesammelt werden – für Unit-Tests sollten daher bewusst kleine, handgeschriebene Beispieldatensätze verwendet werden, wie auch im Kurs gezeigt.

Quelle: [pyspark.testing.assertDataFrameEqual – Apache-Spark-Dokumentation](https://docs.databricks.com/testing/assertDataFrameEqual.html)